If an exception is raised to the left of a rescue modifier, the expression on the left is abandoned, and the value of the overall line is the value of the statement on the right:

```
values = ["1", "2.3", /pattern/]
result = values.map { |v| Integer(v) rescue Float(v) rescue String(v) }
result    # => [1, 2.3, "(?-mix:pattern)"]
```

### Retrying a Block

The retry statement can be used within a rescue clause to restart the enclosing begin/end block from the beginning.

## Catch and Throw

The method catch executes its associated block:

```
catch ( object ) do
  code...
end
```

The method throw interrupts the normal processing of statements:

```
throw( object ‹, obj› )
```

When a throw is executed, Ruby searches up the call stack for the first catch block with a matching object. If it is found, the search stops, and execution resumes past the end of the catch's block. If the throw is passed a second parameter, that value is returned as the value of the catch. Ruby honors the ensure clauses of any block expressions it traverses while looking for a corresponding catch.

If no catch block matches the throw, Ruby raises an ArgumentError exception at the location of the throw.

## Typed Ruby

RBS has its own syntax for defining Ruby types. The definitive source for the syntax reference is https://github.com/ruby/rbs/blob/master/docs/syntax.md. We'll go over the structure here.

### RBS Declarations and File Structure

RBS syntax has a few main parts (we're adopting the naming convention from the official syntax file):

- *Declarations* are class, module, and interface structures that can be used as types in other parts of the file.
- *Members* are things that are inside declarations that get type information, including instance variables, attributes, and methods.
- *Types* are the things that RBS uses to specify actual type information, including class names, union of class names, or literal types, as well as a few RBS-specific keywords.

An RBS file looks like the skeleton of a Ruby file. It declares classes and methods, but does not include the body of methods. You would typically have more than one class in a single RBS file (one namespace per file is common), and I'd say it's rare for an RBS-using application to have top-level methods included in the RBS file.